

kf2vec Tutorial

Overview

kf2vec (“k-mer frequency to vector”) is a tool for converting genome sequences into k-mer feature representations and using them for phylogenetic placement, classification, and model-based distance estimation.

<https://github.com/noraracht/kf2vec>

Installation

Option 1: Bioconda (recommended)

```
conda create -n kf2vec_env python=3.11
conda activate kf2vec_env
conda install -c bioconda kf2vec
```

Option 2: From environment file

```
conda env create -f kf2vec_osx64_v2.yml -n kf2vec_env
conda activate kf2vec_env
```

Basic Usage

Check installation:

```
kf2vec --help
kf2vec --version
```

Or:

```
python -m kf2vec.main --help
```

Step 1: Compute k-mer Frequencies

Convert genome sequences to k-mer vectors:

```
kf2vec get_frequencies -input_dir INPUT_DIR -output_dir OUTPUT_DIR
```

Input: FASTA/FASTQ files (.fa, .fasta, .fq, etc.)

Output: .kf files (k-mer frequency vectors)

Optional:

- -k k-mer size (default 7)
- -p number of processors
- -pseudocount adds smoothing
- -raw_cnt outputs raw counts

Step 2: Split Tree into Subtrees (recommended for large trees)

```
kf2vec divide_tree -size 850 -tree INPUT_PHYLOGENY.nwk
```

Output: .subtrees file assigning each genome to a subtree

Step 3: Train Classifier (optional)

```
kf2vec train_classifier \  
-input_dir INPUT_KF \  
-subtrees FILE.subtrees \  
-e 2000 \  
-o OUTPUT_DIR
```

Output: classifier_model.ckpt

Step 4: Classify Query Sequences

```
kf2vec classify \  
-input_dir QUERY_KF \  
-model MODEL_DIR \  
-o OUTPUT_DIR
```

Output: classes.out (subtree assignments + probabilities)

Step 5: Train Distance Models (main model)

```
kf2vec train_model_set \  
  -input_dir INPUT_KF \  
  -true_dist DIST_DIR \  
  -subtrees FILE.subtrees \  
  -e 4000 \  
  -o OUTPUT_DIR
```

Output: subtree-specific trained models

Step 6: Query Placement / Distance Prediction

```
kf2vec query \  
  -input_dir QUERY_KF \  
  -model MODEL_DIR \  
  -classes CLASSES_DIR \  
  -o OUTPUT_DIR
```

Output: query-to-backbone distance matrices

(Optional) Full Workflow Example

Toy dataset included in repo:

```
kf2vec get_frequencies -input_dir toy_example/train_tree_fna -output_dir  
toy_example/train_tree_kf  
kf2vec get_frequencies -input_dir toy_example/test_fna -output_dir  
toy_example/test_kf  
  
kf2vec divide_tree -tree toy_example/train_tree_newick/train_tree.nwk -size 2  
kf2vec get_distances -tree toy_example/train_tree_newick/train_tree.nwk -  
subtrees toy_example/train_tree_newick/train_tree.subtrees  
  
kf2vec train_classifier -input_dir toy_example/train_tree_kf -subtrees  
toy_example/train_tree_newick/train_tree.subtrees -e 10 -o toy_example/models  
  
kf2vec classify -input_dir toy_example/test_kf -model toy_example/models -o  
toy_example/results
```

Notes

- Requires k-mer preprocessing via Jellyfish (handled internally)
- Works best with high-quality reference genomes
- Large trees should be split using `divide_tree`
- Compatible with downstream placement tools (APPLES + GAPPa)